

Gestión De Empleados Y Nomina.
Informe Sistemas Transaccionales Y Base De Datos.

Presentado Por:

Vanessa María Bastidas

Juan Pablo Lucero Morales

Sebastián Rojas Betancourt

Ubeimar Lizardo Yepes Portada

Presentado A:

Docente Francisco Rojas

Universidad Cooperativa De Colombia.

Ing. De Software – Campus Pasto.

11/2025

Parte 2 – Arquitectura Del Software.

2.1. Objetivo.

Este informe describe de forma detallada cómo se organizaría la aplicación que utiliza la base de datos de gestión de empleados y nómina, asegurando claridad en la estructura, mantenibilidad del código y seguridad en el manejo de información sensible. El documento presenta una arquitectura por capas que separa responsabilidades, facilita el mantenimiento y permite la escalabilidad del sistema.

2.2. Contenido.

1. Introducción Y Objetivo Del Sistema.

El Sistema de Gestión de Empleados y Nómina es una solución integral diseñada para administrar de manera eficiente y segura la información de los empleados de una organización, así como los procesos relacionados con la nómina mensual. El sistema tiene como propósito principal:

Centralizar la información de empleados: Mantener un registro único y consistente de todos los empleados, sus datos personales, laborales y estados dentro de la organización.

Gestionar la estructura organizacional: Administrar departamentos y la asignación de empleados a cada uno, facilitando la organización jerárquica de la empresa.

Controlar aumentos salariales: Registrar y rastrear todos los aumentos salariales aplicados a los empleados, manteniendo un historial completo con fechas, montos anteriores y nuevos, y motivos de los ajustes.

Generar nóminas mensuales: Calcular y registrar las nóminas de cada empleado por período, incluyendo horas extra trabajadas, devengados, deducciones y el monto neto a pagar.

Proveer reportes operativos: Generar consultas y reportes que permitan tomar decisiones informadas sobre la gestión del personal y los costos laborales.

Alcance Funcional.

El sistema cubre las siguientes funcionalidades principales:

Gestión de Departamentos:

- Registro de nuevos departamentos
- Actualización de información de departamentos existentes
- Consulta de departamentos y sus empleados asociados

Gestión de Empleados

- Registro de nuevos empleados con validación de datos únicos (documento, email).
- Actualización de información personal y laboral.
- Gestión de estados del empleado: activo, inactivo, suspendido.
- Asignación de empleados a departamentos.
- Consulta de empleados por diferentes criterios (departamento, estado, fecha de contratación).

Gestión de Aumentos Salariales

- Registro de aumentos salariales con validación de integridad
- Actualización automática del salario actual del empleado
- Trazabilidad completa: fecha, salario anterior, nuevo salario y motivo
- Validación de que el salario anterior coincida con el actual antes de aplicar el aumento
- Prevención de múltiples aumentos en la misma fecha para un mismo empleado

Generación de Nóminas

- Generación de nómina mensual por empleado
- Validación de que el empleado esté activo antes de generar nómina
- Registro de horas extra trabajadas en el período
- Prevención de nóminas duplicadas para el mismo empleado en el mismo período (año/mes)
- Consulta histórica de nóminas generadas

Reportes y Consultas

- Reporte de horas extra por mes, ordenado por cantidad
- Promedio salarial por departamento (solo empleados activos)
- Historial completo de aumentos salariales con cálculo de incrementos y porcentajes
- Consulta de contrataciones recientes (últimos 6 meses).

2. Arquitectura General.

Estilo Arquitectónico Propuesto.

Se propone una arquitectura por capas (monolito modular) que separa claramente las responsabilidades del sistema en tres capas principales:

Capa de Presentación / API: Maneja la interacción con los usuarios o sistemas externos.

Capa de Dominio / Lógica: Contiene las reglas de negocio y la lógica de procesamiento.

Capa de Datos / Persistencia: Gestiona el acceso y almacenamiento de datos.

Esta arquitectura facilita:

Mantenibilidad: Cambios en una capa no afectan directamente a las otras.

Testabilidad: Cada capa puede ser probada de forma independiente.

Escalabilidad: Permite optimizar cada capa según sus necesidades específicas.

Claridad: Cada componente tiene una responsabilidad bien definida.

Diagrama De Arquitectura.



Módulos o componentes principales y sus responsabilidades.

Capa de Presentación / API

- Responsabilidades:

Recibir y validar las peticiones de los clientes (aplicaciones web, móviles, sistemas externos)

Transformar los datos de entrada en DTOs (Data Transfer Objects) para la capa de dominio

Aplicar autenticación y autorización basada en roles

Validar formato y tipos de datos de entrada

Transformar las respuestas de la capa de dominio en formatos adecuados (JSON, XML)

- Manejar errores y devolver códigos HTTP apropiados

- Componentes principales:

EmpleadoController / EmpleadoAPI

`POST /api/empleados` - Registrar nuevo empleado

`GET /api/empleados` - Listar empleados (con paginación y filtros)

`GET /api/empleados/{id}` - Obtener empleado por ID

`PUT /api/empleados/{id}` - Actualizar empleado

`PATCH /api/empleados/{id}/estado` - Cambiar estado del empleado

`GET /api/empleados/{id}/aumentos` - Obtener historial de aumentos

`GET /api/empleados/recientes` - Obtener contrataciones recientes

DepartamentoController / DepartamentoAPI

`POST /api/departamentos` - Crear departamento

`GET /api/departamentos` - Listar departamentos

`GET /api/departamentos/{id}/empleados` - Empleados por departamento

`GET /api/departamentos/promedio-salarios` - Promedio salarial por departamento

Gestión De Empleados Y Nomina.

NominaController / NominaAPI

- `POST /api/nominas/generar` - Generar nómina mensual para un empleado
- `GET /api/nominas` - Listar nóminas (con filtros por período, empleado)
- `GET /api/nominas/{id}` - Obtener nómina por ID
- `GET /api/nominas/horas-extra` - Reporte de horas extra por mes
- `GET /api/nominas/periodo/{anio}/{mes}` - Nóminas de un período específico

AumentoController / AumentoAPI

- `POST /api/aumentos` - Registrar aumento salarial
- `GET /api/aumentos` - Listar aumentos (con filtros)
- `GET /api/aumentos/empleado/{id}` - Aumentos de un empleado específico
- `GET /api/aumentos/historial` - Historial completo de aumentos

Validaciones de entrada

- Validación de formato de email
- Validación de formato de documento (según país)
- Validación de rangos numéricos (salarios > 0, períodos válidos)
- Validación de estados permitidos
- Validación de unicidad de documento y email

Capa de Dominio / Lógica

- Responsabilidades
 - Implementar las reglas de negocio del sistema
 - Validar la coherencia de los datos antes de persistirlos
 - Realizar cálculos complejos (salarios, deducciones, devengados)
 - Coordinar operaciones transaccionales que involucran múltiples entidades
 - Mantener la integridad de los datos a nivel de lógica de negocio

- Componentes principales

NominaService

Calcular total devengado: Suma del salario base más el valor de las horas extra

```
```python
total_devengado = salario_base + (horas_extra * valor_hora_extra)
```

Calcular total deducciones: Suma de todas las deducciones aplicables (salud, pensión, retención en la fuente, etc.)

```
```python
total_deducciones = deduccion_salud + deduccion_pension + retencion_fuente + otros
...

```

Calcular neto pagado: Diferencia entre devengado y deducciones

```
```python
neto_pagado = total_devengado - total_deducciones
...

```

Validar empleado activo: Verificar que el empleado esté en estado 'activo' antes de generar nómina

Validar período único: Asegurar que no exista una nómina previa para el mismo empleado en el mismo período

Aplicar reglas de horas extra: Validar límites máximos de horas extra según legislación laboral

#### AumentoService

Validar salario anterior: Verificar que el salario anterior registrado coincida con el salario actual del empleado

Aplicar aumento: Actualizar el salario actual del empleado después de registrar el aumento

Validar incremento positivo: Asegurar que el nuevo salario sea mayor que el anterior

Registrar motivo: Almacenar el motivo del aumento para trazabilidad

Prevenir duplicados: Evitar múltiples aumentos en la misma fecha para un mismo empleado

#### EmpleadoService

Validar estados: Gestionar las transiciones válidas entre estados (activo → inactivo, activo → suspendido, etc.)

Validar unicidad: Verificar que documento y email sean únicos antes de crear o actualizar

Gestionar asignación a departamento: Validar que el departamento exista antes de asignar empleado

Calcular antigüedad: Calcular tiempo de servicio basado en fecha de contratación

#### ReporteService

Generar reporte de horas extra: Agrupar y ordenar horas extra por mes

Calcular promedios salariales: Agregar salarios por departamento para empleados activos

Generar historial de aumentos: Consolidar información de aumentos con cálculos de incrementos y porcentajes

Filtrar contrataciones recientes: Aplicar filtro de fecha para últimos 6 meses

Objetos de Dominio:

`Empleado`: Entidad que representa un empleado con sus atributos y comportamientos

`Departamento`: Entidad que representa un departamento organizacional

`Nomina`: Entidad que representa una nómina mensual con sus cálculos

`Aumento`: Entidad que representa un aumento salarial con su trazabilidad

`ValidadorEstado`: Clase que valida transiciones de estado

`CalculadoraNomina`: Clase que realiza cálculos de nómina

### **Capa de Datos / Persistencia**

- Responsabilidades

Abstraer el acceso a la base de datos

Implementar consultas optimizadas y parametrizadas

Gestionar transacciones para operaciones complejas

Mapear objetos de dominio a estructuras de base de datos

Implementar caché cuando sea apropiado

- Componentes principales

EmpleadoRepository

`create(Empleado)` : Insertar nuevo empleado

`findById(id)` : Buscar empleado por ID

`findByDocumento(documento)` : Buscar por documento único

`findByEmail(email)` : Buscar por email único

`findByDepartamento(departamentoid)` : Listar empleados de un departamento

`findByEstado(estado)` : Listar empleados por estado

`findContratacionesRecientes(meses)` : Empleados contratados en últimos N meses

`update(empleado)` : Actualizar información del empleado

`updateSalario(empleadold, nuevoSalario)` : Actualizar solo el salario

#### NominaRepository

`create(nomina)` : Insertar nueva nómina

`findByld(id)` : Buscar nómina por ID

`findByEmpleado(empleadold)` : Historial de nóminas de un empleado

`findByPeriodo(anio, mes)` : Nóminas de un período específico

`findByEmpleadoYPeriodo(empleadold, anio, mes)` : Verificar existencia de nómina

`findHorasExtraPorMes(anio, mes)` : Reporte de horas extra ordenado

`exists(empleadold, anio, mes)` : Verificar si existe nómina para período

#### AumentoRepository

`create(aumento)` : Insertar nuevo aumento

`findByld(id)` : Buscar aumento por ID

`findByEmpleado(empleadold)` : Historial de aumentos de un empleado

`findByFecha(empleadold, fecha)` : Verificar aumento en fecha específica

`findAll()` : Listar todos los aumentos con información de empleados

#### DepartamentoRepository

`create(departamento)` : Crear nuevo departamento

`findByld(id)` : Buscar departamento por ID

`findByNombre(nombre)` : Buscar por nombre único

`findAll()` : Listar todos los departamentos

``findConPromedioSalarial()`: Departamentos con promedio salarial de empleados activos`

#### Características de implementación

Consultas parametrizadas: Todas las consultas usan parámetros para prevenir inyección SQL

Transacciones: Operaciones complejas se ejecutan dentro de transacciones

Conexión pooling: Gestión eficiente de conexiones a la base de datos

### 3. Patrones Y Estructuras Utilizadas

#### Patrón Repository

El patrón Repository se utiliza para abstraer la lógica de acceso a datos y proporcionar una interfaz más orientada a objetos para acceder a la capa de persistencia.

#### Implementación

Cada entidad principal tiene su propio repositorio: ``EmpleadoRepository``, ``NominaRepository``, ``AumentoRepository``, ``DepartamentoRepository``

Los repositorios encapsulan todas las consultas SQL relacionadas con una entidad

Proporcionan métodos de alto nivel que la capa de dominio puede usar sin conocer detalles de SQL

#### DTOs (Data Transfer Objects)

Los DTOs se utilizan para transferir datos entre capas sin exponer las entidades de dominio directamente.

#### DTOs principales

1. EmpleadoDTO

```
```json
{
  "id": 1,
  "nombre": "Ana",
  "apellido": "Pérez",
  "documento": "CC1001",
  "email": "ana.perez@empresa.com",
  "telefono": "3001112223",
  "fechaContratacion": "2024-02-15",
  "estado": "activo",
  "salarioActual": 3700000,
  "departamentold": 1,
  "departamentoNombre": "Finanzas"
}
```
```

## 2. NominaDTO

```
```json
{
  "id": 1,
  "empleadold": 1,
  "empleadoNombre": "Ana Pérez",
  "periodoAnio": 2025,
  "periodoMes": 11,
  "horasExtra": 8.0,
  "salarioBase": 3700000,
  "totalDevengado": 3820000,
  "totalDeducciones": 380000,
  "netoPagado": 3440000,
}
```

```
"fechaGeneracion": "2025-11-30"  
}  
...
```

3. AumentoDTO

```
```json  
{
 "id": 1,
 "empleadold": 1,
 "empleadoNombre": "Ana Pérez",
 "fecha": "2025-03-01",
 "salarioAnterior": 3500000,
 "nuevoSalario": 3700000,
 "incremento": 200000,
 "porcentaje": 5.71,
 "motivo": "Desempeño"
}
...
```

### Validaciones

Las validaciones se implementan en múltiples capas para asegurar la integridad de los datos.

Nivel de API

Validación de formato (email, teléfono, fechas)

Validación de tipos de datos

Validación de rangos numéricos

Validación de campos requeridos

#### Nivel de Dominio

Validación de reglas de negocio (estado del empleado, unicidad)

Validación de coherencia (salario anterior vs actual)

Validación de transiciones de estado permitidas

#### Nivel de Base de Datos

Constraints CHECK para rangos y valores permitidos

Constraints UNIQUE para garantizar unicidad

Foreign Keys para integridad referencial

Triggers para validaciones complejas

#### Ejemplo de validación en dominio

```
```java
```

```
public void validarAumento(Aumento aumento) {  
    Empleado empleado = empleadoRepository.findById(aumento.getEmpleadId());  
  
    if (empleado == null) {  
        throw new EmpleadoNoEncontradoException();  
    }  
  
    if (!empleado.getSalarioActual().equals(aumento.getSalarioAnterior())) {  
        throw new SalarioAnteriorNoCoincideException();  
    }  
  
    if (aumento.getNuevoSalario().compareTo(aumento.getSalarioAnterior()) <= 0) {  
        throw new IncrementoInvalidoException();  
    }  
}
```

Transacciones

Las transacciones garantizan la atomicidad de operaciones complejas que involucran múltiples pasos.

Operaciones transaccionales principales

Generación de Nómina

```
```java
@Transactional
public Nomina generarNomina(Long empleadoid, int anio, int mes,
 BigDecimal horasExtra) {
 // 1. Validar empleado activo
 // 2. Verificar que no exista nómina para el período
 // 3. Calcular devengados, deducciones y neto
 // 4. Insertar nómina
 // 5. Registrar auditoría
 // Si cualquier paso falla, se revierte todo
}
```

Aplicación de Aumento

```
```java
@Transactional
public Aumento aplicarAumento(Aumento aumento) {
    // 1. Validar salario anterior
    // 2. Insertar registro de aumento
    // 3. Actualizar salario actual del empleado
    // 4. Registrar auditoría
    // Si cualquier paso falla, se revierte todo
}
```

...

Características

Uso de '@Transactional' o equivalente para demarcar transacciones

Rollback automático en caso de excepciones

Aislamiento de transacciones para prevenir condiciones de carrera

Servicios de Dominio

Los servicios de dominio encapsulan lógica de negocio que no pertenece naturalmente a una entidad específica.

Servicios implementados

NominaService: Contiene la lógica de cálculo de nóminas

AumentoService: Contiene la lógica de aplicación de aumentos

ReporteService: Contiene la lógica de generación de reportes

ValidacionService: Contiene validaciones complejas reutilizables

4. Seguridad Y Manejo De Errores.

Validacion de entradas.

Todas las entradas del sistema son validadas antes de ser procesadas para prevenir datos inválidos o maliciosos.

Validaciones implementadas

Validación de formato

Email: Expresión regular para formato válido

Documento: Formato según tipo (CC, NIT, etc.)

Teléfono: Formato numérico con longitud válida

Fechas: Formato ISO 8601 (YYYY-MM-DD)

Validación de rangos

Salarios: Debe ser mayor que 0

Períodos: Año entre 2000-2100, mes entre 1-12

Horas extra: Debe ser mayor o igual a 0

Porcentajes: Entre 0 y 100

Validación de estados

Estado de empleado: Solo valores permitidos ('activo', 'inactivo', 'suspendido')

Validación de transiciones de estado permitidas

Validación de unicidad

Documento: Debe ser único en el sistema

Email: Debe ser único en el sistema

Nómina: Un empleado no puede tener dos nóminas en el mismo período

Aumento: Un empleado no puede tener dos aumentos en la misma fecha

Consultas Parametrizadas

Todas las consultas a la base de datos utilizan parámetros para prevenir inyección SQL.

Ejemplo de consulta parametrizada

```
``java
```

```
// INCORRECTO - Vulnerable a inyección SQL
```

```
String sql = "SELECT * FROM empleado WHERE documento = " + documento + "";
```

```
// CORRECTO - Consulta parametrizada
```

```
String sql = "SELECT * FROM empleado WHERE documento = ?";
```

```
PreparedStatement stmt = connection.prepareStatement(sql);
```

```
stmt.setString(1, documento);
```

Manejo de Excepciones

El sistema implementa un manejo de excepciones estructurado que proporciona mensajes claros y códigos HTTP apropiados.

Jerarquía de excepciones

Excepciones de negocio

- `EmpleadoNoEncontradoException`: Empleado no existe
- `EmpleadoNoActivoException`: Empleado no está activo
- `NominaDuplicadaException`: Ya existe nómina para el período
- `SalarioAnteriorNoCoincideException`: Salario anterior no coincide
- `DocumentoDuplicadoException`: Documento ya existe
- `EmailDuplicadoException`: Email ya existe

Excepciones de validación

- `ValidacionException`: Error de validación de datos
- `FormatoInvalidoException`: Formato de dato inválido

Excepciones de sistema

- `ErrorBaseDatosException`: Error de conexión o consulta
- `ErrorTransaccionException`: Error en transacción

Manejo centralizado

```
```java
```

```
@ControllerAdvice
```

```
public class GlobalExceptionHandler {

 @ExceptionHandler(EmpleadoNoEncontradoException.class)
 public ResponseEntity<ErrorResponse> handleEmpleadoNoEncontrado(
 EmpleadoNoEncontradoException ex) {
 ErrorResponse error = new ErrorResponse(
 "EMPLEADO_NO_ENCONTRADO",
 ex.getMessage(),
 HttpStatus.NOT_FOUND.value()
);
 return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
 }

 @ExceptionHandler(ValidacionException.class)
 public ResponseEntity<ErrorResponse> handleValidacion(ValidacionException ex) {
 ErrorResponse error = new ErrorResponse(
 "VALIDACION_ERROR",
 ex.getMessage(),
 HttpStatus.BAD_REQUEST.value()
);
 return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
 }
}
```

Formato de respuesta de error

```
``json
{
 "codigo": "EMPLEADO_NO_ENCONTRADO",
```

```
"mensaje": "No se encontró un empleado con el ID proporcionado",
"status": 404,
"timestamp": "2025-01-15T10:30:00Z"
}
```

## 5. Optimización Y Escalabilidad.

Se han creado índices estratégicos en las columnas más consultadas para optimizar el rendimiento de las consultas.

Índices implementados

Tabla empleada

```
```sql  
  
CREATE INDEX idx_empleado_departamento ON empleado(departamento_id);  
-- Optimiza: Consultas de empleados por departamento  
  
CREATE INDEX idx_empleado_estado ON empleado(estado);  
-- Optimiza: Filtros por estado (activo, inactivo, suspendido)  
  
CREATE INDEX idx_empleado_fecha_contratacion ON empleado(fecha_contratacion);  
-- Optimiza: Consultas de contrataciones recientes  
...  
  
- **Índice único implícito**: `documento` y `email` (por constraint UNIQUE)  
- **Índice primario**: `id` (automático)
```

Tabla nomina

```
```sql  

CREATE INDEX idx_nomina_empleado_periodo ON nomina(empleado_id,
periodo_anio, periodo_mes);
```

## Gestión De Empleados Y Nomina.

Optimiza: Búsqueda de nóminas por empleado y período

Índice compuesto para consultas frecuentes

Índice primario: `id`

Índice único compuesto\*\*: `(empleado\_id, periodo\_anio, periodo\_mes)` para prevenir duplicados

### Tabla aumento

sql

CREATE INDEX idx\_aumento\_empleado\_fecha ON aumento(empleado\_id, fecha);

Optimiza: Historial de aumentos por empleado ordenado por fecha

Índice primario: `id`

Índice único compuesto: `(empleado\_id, fecha)` para prevenir duplicados

## Vistas y Materializaciones para Reportes

Se utilizan vistas para simplificar consultas complejas y materializaciones para reportes frecuentes.

### Vistas propuestas

#### Vista de horas extra por mes

``sql

CREATE VIEW v\_horas\_extra\_mes AS

SELECT

n.periodo\_anio,

n.periodo\_mes,

## Gestión De Empleados Y Nomina.

```
e.id AS empleado_id,
e.nombre || ' ' || e.apellido AS empleado_nombre,
n.horas_extra,
ROW_NUMBER() OVER (
 PARTITION BY n.periodo_anio, n.periodo_mes
 ORDER BY n.horas_extra DESC
) AS ranking
FROM nomina n
JOIN empleado e ON e.id = n.empleado_id
WHERE e.estado = 'activo';
```

Uso: Reporte de ranking de horas extra

Beneficio: Consulta simplificada y reutilizable

## Vista de promedios salariales por departamento

```
``sql
CREATE VIEW v_promedio_salario_departamento AS
SELECT
 d.id AS departamento_id,
 d.nombre AS departamento,
 COUNT(e.id) AS cantidad_empleados,
 AVG(e.salario_actual) AS promedio_salario,
 MIN(e.salario_actual) AS salario_minimo,
 MAX(e.salario_actual) AS salario_maximo
FROM departamento d
LEFT JOIN empleado e ON e.departamento_id = d.id AND e.estado = 'activo'
GROUP BY d.id, d.nombre;
```

## Gestión De Empleados Y Nomina.

Uso: Reporte de estadísticas salariales

Beneficio: Cálculo pre-agregado para consultas rápidas

### Vista materializada de historial de aumentos

```
```sql
CREATE MATERIALIZED VIEW mv_historial_aumentos AS
SELECT
    a.id,
    e.id AS empleado_id,
    e.nombre || ' ' || e.apellido AS empleado_nombre,
    a.fecha,
    a.salario_anterior,
    a.nuevo_salario,
    (a.nuevo_salario - a.salario_anterior) AS incremento,
    ROUND(((a.nuevo_salario / a.salario_anterior) - 1) * 100, 2) AS
porcentaje_incremento,
    a.motivo
FROM aumento a
JOIN empleado e ON e.id = a.empleado_id;

CREATE INDEX ON mv_historial_aumentos(empleado_id, fecha);
```

Uso: Reporte de historial de aumentos con cálculos pre-computados

Beneficio: Consultas muy rápidas, actualización periódica según necesidad

Paginación en Listados

Todos los endpoints que devuelven listas implementan paginación para evitar cargar grandes volúmenes de datos.

Implementación de paginación

```
```java
@GetMapping("/api/empleados")
public ResponseEntity<Page<EmpleadoDTO>> listarEmpleados(
 @RequestParam(defaultValue = "0") int page,
 @RequestParam(defaultValue = "20") int size,
 @RequestParam(required = false) String estado,
 @RequestParam(required = false) Long departamentold) {

 Pageable pageable = PageRequest.of(page, size);

 Page<Empleado> empleados = empleadoService.findAll(pageable, estado,
departamentold);

 Page<EmpleadoDTO> dtos = empleados.map(this::toDTO);

 return ResponseEntity.ok(dtos);
}
```
```

****Respuesta paginada:****

```
```json
{
 "content": [
 {
 "id": 1,
 "nombre": "Ana",

```

```
"apellido": "Pérez",
...
}
],
"page": {
 "number": 0,
 "size": 20,
 "totalElements": 150,
 "totalPages": 8
}
}
```

## 6. Conclusiones técnicas

El desarrollo de este sistema permitió adquirir aprendizajes técnicos fundamentales para la construcción de software robusto, mantenible y escalable. La correcta separación por capas evidenció la importancia de distribuir responsabilidades para facilitar la comprensión del código, mejorar la capacidad de prueba y permitir la reutilización de la lógica de negocio desde diferentes interfaces. De igual forma, el diseño cuidadoso de la base de datos, con sus restricciones, claves y triggers, demostró ser una herramienta decisiva para garantizar la integridad y consistencia de la información, especialmente en procesos sensibles como la generación de nómina.

Asimismo, la creación estratégica de índices y la aplicación del patrón Repository aseguraron un sistema más eficiente y flexible, permitiendo una capa de persistencia desacoplada y optimizada. El manejo adecuado de transacciones también resultó clave para mantener la coherencia del sistema, evitando estados intermedios o inconsistentes ante fallos inesperados. Todos estos aspectos combinados contribuyeron al fortalecimiento del diseño y la operación del sistema.

No obstante, el proyecto también evidenció limitaciones importantes que abren oportunidades de mejora. Algunas validaciones complejas dependen de múltiples entidades relacionadas, lo que ha llevado a soluciones basadas en triggers que, aunque

funcionales, pueden complicar la depuración y ocultar lógica crítica. De igual forma, la estructura actual no permite registrar horas extra detalladas por día ni desglosar las deducciones, lo que limita los análisis y auditorías. Tampoco existe un historial de cambios en empleados, lo que dificulta procesos de trazabilidad y cumplimiento. Finalmente, la arquitectura monolítica restringe la escalabilidad horizontal, lo cual impacta la proyección a largo plazo del sistema.

Frente a esto, se plantean mejoras futuras que fortalecerían ampliamente las capacidades del sistema. La integración con un módulo de asistencia permitiría automatizar el cálculo de horas extra; la exportación a ERPs facilitaría procesos contables; y la creación de un módulo de vacaciones y ausentismo centralizaría la gestión del tiempo laboral. Un dashboard analítico impulsaría la toma de decisiones basada en datos, mientras que la implementación de notificaciones automáticas optimizaría la comunicación con empleados y administrativos. Finalmente, la creación de una API de integración permitiría ampliar el ecosistema del sistema, conectándolo con bancos, proveedores y plataformas externas.

En general, el proyecto deja como resultado un sistema funcional y técnicamente sólido, con bases arquitectónicas bien definidas y una estructura que permite seguir evolucionando. Las oportunidades de mejora identificadas son un camino claro hacia un sistema más completo, automatizado y escalable, preparado para integrarse con otros servicios y responder a necesidades organizacionales en crecimiento.

